

Typing Rush: A Terminal-Based Typing Tutor Leveraging Operating System Concepts

Kush Agarwal (230050)
Rishihood University

Himanshu Rawat (230096)
Rishihood University

Abstract—This report presents the final Phase 2 implementation of *Typing Rush*, a terminal-based typing application developed in C using five custom libraries: math, string, memory, screen, and keyboard. Phase 1 established the core integration and real-time typing loop. Phase 2 extends that baseline into a complete, playable system with structured game flow, multiple game modes, persistent high-score storage, improved runtime stability, and robust error handling. The implementation preserves low-level design principles and uses custom logic for memory, string processing, bounds checking, and screen handling. The final system is stable for normal use, responsive during play, and complete from start to finish.

Index Terms—C programming, terminal application, custom libraries, systems programming, memory management

I. INTRODUCTION

The objective of this capstone project is to design and build a complete software system in C from scratch using self-developed foundational libraries. The selected track is an interactive terminal application. The final deliverable is *Typing Rush*, a real-time typing game that demonstrates custom library integration, process-aware control flow, and stable runtime behavior.

In Phase 1, the project implemented baseline integration: a working real-time loop, non-blocking keyboard input, typed-text rendering, and dynamic memory allocation/deallocation. In Phase 2, the same architecture was expanded into a complete application with structured navigation, gameplay modes, result handling, and persistent score storage.

II. METHODOLOGY

A. Design Approach

The system was developed incrementally to reduce integration risk and maintain stability at each stage:

- 1) Build and validate all five custom libraries independently.
- 2) Integrate the libraries in a basic playable loop (Phase 1 baseline).
- 3) Extend the baseline into a full game-state flow for Phase 2.
- 4) Add persistence, robustness checks, and UI stability improvements.
- 5) Re-test game completion paths, input edge cases, and terminal behavior.

B. System Architecture

The application follows a modular pipeline:

- **keyboard.c** handles raw-mode non-blocking input.
- **string.c** handles string utilities required by gameplay and UI.
- **math.c** handles integer arithmetic, clamping, and bounds checks.
- **memory.c** handles virtual RAM allocation and deallocation.
- **screen.c** handles terminal rendering and cursor control.

This modular architecture keeps each responsibility isolated and improves maintainability.

C. Phase 1 Baseline

Phase 1 provided the foundational working version with:

- one active typing lesson,
- real-time key processing,
- colored correctness overlay,
- dynamic memory allocation for runtime buffers,
- clean terminal enter/exit behavior.

This baseline was retained and extended, not replaced.

D. Phase 2 Implementation

Phase 2 introduced complete system behavior:

- state-driven game flow (menu, mode selection, gameplay, name entry, results, highscores),
- multiple gameplay modes (Practice, Timed 30s, Timed 60s, Survival),
- difficulty-based lesson selection,
- score, accuracy, and WPM computation,
- high-score persistence with player-name input,
- safe handling for interrupt and terminal resize signals.

E. Implementation Notes for Stability

To avoid runtime issues, the implementation applies:

- strict bounds checks before writing typed input,
- validation checks before file read/write operations,
- guaranteed deallocation on major game-exit paths,
- controlled redraw logic for resize events,
- deterministic state transitions.

These measures improved consistency and reduced UI/runtime breakage during testing.

III. RESULTS AND DISCUSSION

The final Phase 2 application is fully playable and completes the expected flow from startup to results and high-score updates. Compared with Phase 1, the system now supports complete game navigation, richer interaction, and persistent user outcomes.

From a systems perspective, the project demonstrates:

- practical use of custom memory management in live gameplay,
- non-blocking terminal I/O in a real-time loop,
- robust input/path handling under normal execution,
- stable shutdown and terminal restoration.

The resulting application meets the capstone objective of full system integration using self-built libraries.

IV. FUTURE SCOPE

The following improvements are suitable for future extension:

- 1) loading custom lessons from external text files,
- 2) adding richer player analytics (error patterns, per-character accuracy),
- 3) introducing checksum/versioning in score files,
- 4) adding automated stress tests for long-running sessions,
- 5) improving adaptive difficulty based on live performance.

V. CONCLUSION

The project successfully progressed from a Phase 1 baseline to a complete Phase 2 application. The final system integrates all required custom libraries into one stable, interactive terminal game. It remains responsive during play, handles key edge cases safely, and provides a full start-to-finish experience with persistent scoring.

REFERENCES

- [1] ISO/IEC 9899:2018, *Programming Languages — C*, 2018.
- [2] The Open Group, *POSIX.1-2017 Base Specifications*, 2018.